Introduction to Error-Correcting Codes

Pierre Fromonot

October 12, 2026

Contents

1 Theory of Error-Correcting Codes **2**
- 1.1 Motivation . . . 2
- 1.2 Codes and Linear Codes . . . 2
  - 1.2.1 Codes . . . 2
  - 1.2.2 Linear Codes . . . 2
  - 1.2.3 Notation . . . 3
- 1.3 Generator and Parity-Check Matrices . . . 3
- 1.4 Syndrome Decoding . . . 3
- 1.5 Distance and Error Correction Capability . . . 4
  - 1.5.1 The Minimum Distance . . . 4
  - 1.5.2 The Error-Correction Bound . . . 4
  - 1.5.3 Geometric Intuition . . . 4
  - 1.5.4 Decoding Algorithm . . . 5
  - 1.5.5 Code Parameters and Trade-offs . . . 5
- 1.6 Systematic Forms of Codes . . . 5
  - 1.6.1 Intuition: Information and Redundancy . . . 6
  - 1.6.2 Systematic Forms Always Exist . . . 6
  - 1.6.3 Relationship with the Parity-Check Matrix . . . 6
- 1.7 Bounds on Code Parameters . . . 7

2 Examples of Codes **9**
- 2.1 Hamming Codes . . . 9
  - 2.1.1 The idea . . . 9
  - 2.1.2 Example: the $[7, 4]$ Hamming code . . . 10
  - 2.1.3 SECDED Codes . . . 10
- 2.2 Reed-Solomon Codes . . . 11
  - 2.2.1 The principle . . . 11
  - 2.2.2 A small Reed–Solomon example . . . 13
- 2.3 Low Density Parity Check (LDPC) Codes . . . 15
  - 2.3.1 Definition and Bit-Flipping Algorithm . . . 15
  - 2.3.2 Example 1: Bit-Flipping that succeeds . . . 17
  - 2.3.3 Example 2: Bit-Flipping that Fails . . . 18
- 2.4 Random Codes . . . 19

3 Error-Correcting Codes in Cryptography **20**
- 3.1 Motivation . . . 20
- 3.2 BIKE Scheme . . . 20

Chapter 1

Theory of Error-Correcting Codes

1.1 Motivation

Error-correcting codes are mathematical objects that, combined with the right algorithms, detect and correct data errors by adding redundancy to transmitted or stored information. This allows systems to recover original data without requiring re-transmission, which is critical in noisy channels, memory storage, and deep-space communications.

1.2 Codes and Linear Codes

1.2.1 Codes

At its core, a *code* is simply a set of allowed sequences (called *codewords*) that we agree upon before communication. The sender chooses a codeword from this set, transmits it through a noisy channel, and the receiver tries to decode it, correcting any errors introduced by noise.

Definition 1.2.1. *A (binary) code of length n is a non-empty subset $C \subseteq \mathbb{F}_2^n$. Each element of C is called a codeword. The rate of the code is $\log_2 |C|/n$.*

Example 1.2.2. *A simple example is the repetition code: to send one bit, repeat it three times. Here $C = \{000, 111\} \subset \mathbb{F}_2^3$. If a single error occurs (e.g., $000 \to 010$), we can still decode by majority vote.*

1.2.2 Linear Codes

In principle, one could study any subset of $\mathbb{F}_2^n$. However, linear codes — codes that form a vector subspace — have special structure that makes them practical:

- **Algebraic structure:** Linear operations (addition, scalar multiplication) are well-defined, allowing us to leverage tools from linear algebra.
- **Efficient encoding:** We can represent the code compactly using a generator matrix G, which encodes any message via a simple matrix-vector product.
- **Efficient decoding:** The structure of linear codes enables fast decoding algorithms, such as syndrome decoding (which we will discuss later).
- **Distance properties:** For linear codes, the Hamming distance between codewords is directly related to the weight of other codewords, simplifying analysis.

Fortunately, many of the best codes discovered are linear. For this reason, we will focus exclusively on linear codes.

1.2.3 Notation

We denote a linear code with parameters n, k, and minimum distance d as an $[n, k, d]$ code, where:

- n is the **length** (number of bits in a codeword),
- k is the **dimension** (log-number of codewords: $|C| = 2^k$),
- d is the **minimum distance** (smallest Hamming distance between distinct codewords; we define this formally later).

1.3 Generator and Parity-Check Matrices

Formally, a linear code C is a vector subspace of $\mathbb{F}_2^n$ of dimension $k \leq n$. We write G a *generating matrix* $(n \times k)$ of C: its rows form a basis of C,

$$C = \{x \cdot G : x \in \mathbb{F}_2^k\}$$

Here x is a *message*, and $c = x \cdot G \in \mathbb{F}_2^n$ is a *codeword*. As $n \geq k$, computing $c = x \cdot G$ adds redundancy to the message x.

When transmitted, $c = x \cdot G$ may be corrupted by an error which can be seen as a vector $e \in \mathbb{F}_2^n$: $y = c + e = x \cdot G + e$. The redundancy added to x can help recover the actual message x in some cases when the code C is well structured and if the error e is not too large (if the error occurs on $t \ll n$ bits, i.e., e has a small Hamming weight).

We define the *dual code* of C as

$$C^\perp = \{v \in \mathbb{F}_2^n : v \cdot G = 0\}$$

This is the left kernel of G. The *parity matrix* H $(m \times n)$ of the code C is defined by

$$H \cdot c^T = 0, \qquad \forall c \in C$$

where $m = n - k$ is the number of *parity bits*. This is a generating matrix of $C^\perp$.

1.4 Syndrome Decoding

Here is why H is important. When a codeword $c \in C$ is transmitted through a noisy channel, the received word is $y = c + e$, where e is the error vector. Although the receiver does not know c or e directly, it can compute the *syndrome*:

$$s = Hy^T = H(c + e)^T = Hc^T + He^T = He^T,$$

since $Hc^T = 0$ for all $c \in C$ (by definition of the parity-check matrix).

The syndrome s is a crucial quantity: it depends only on the error e, not on the message or codeword. We can think of the syndrome as a signature of the error. Given the syndrome, the decoder's goal is to determine which error e most likely occurred, and thus recover the original codeword as $c = y - e$.

For this strategy to work, we need the error to be small enough. In particular, if two distinct errors e_1 and e_2 produce the same syndrome, then the decoder cannot distinguish between them, and decoding fails. This motivates the concept of the minimum distance, which tells us the maximum number of errors that can always be corrected uniquely.

1.5 Distance and Error Correction Capability

1.5.1 The Minimum Distance

For a linear code C, the *minimum distance* is defined as the smallest Hamming distance between two distinct codewords:

$$d := \min_{\substack{c_1,c_2 \in C \\ c_1 \neq c_2}} d_H(c_1, c_2).$$

A key property of linear codes is that this is equivalent to the minimum weight of non-zero codewords:

$$d = \min_{c \in C \setminus \{0\}} w_H(c).$$

This holds because $d_H(c_1, c_2) = w_H(c_1 - c_2) = w_H(c_1 + c_2)$ (in $\mathbb{F}_2$), and $c_1 + c_2 \in C$ whenever $c_1, c_2 \in C$.

1.5.2 The Error-Correction Bound

The minimum distance directly determines how many errors a code can correct. The intuition is geometric: if we place a ball of radius r around each codeword, and these balls are disjoint, then any received word within distance r of a codeword can be uniquely decoded to that codeword.

Theorem 1.5.1. *Let C be a linear $[n, k, d]$ code. Then C can correct up to*

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor$$

errors. That is, if e is an error vector with $w_H(e) \leq t$ and $c \in C$ is the transmitted codeword, then c is the unique codeword closest to $y = c + e$.

Proof. Let $y = c + e$ be the received word, where c is the transmitted codeword and e is an error of weight $w_H(e) \leq t$. We need to show that c is the unique codeword closest to y. The distance from y to any codeword c' is

$$d_H(y, c') = d_H(c + e, c') = w_H(c + e + c').$$

If $d_H(y, c') \leq t$, then $w_H(c+e+c') \leq t$. Now, $c+c'$ is a codeword (as the difference of two codewords), and let's write $c + c' =: \tilde{c}$. Then

$$w_H(\tilde{c} + e) \leq t.$$

By the triangle inequality for Hamming weight,

$$w_H(\tilde{c}) \leq w_H(\tilde{c} + e) + w_H(e) \leq t + t = 2t.$$

But $\tilde{c} = c + c'$ is a non-zero codeword (since $c \neq c'$), so $w_H(\tilde{c}) \geq d$ by definition of minimum distance. Therefore, $d \leq 2t$. However, $t = \lfloor (d-1)/2 \rfloor$, which means $t \leq (d-1)/2$, so $2t \leq d - 1 < d$. This is a contradiction.

Thus, there cannot exist another codeword $c' \neq c$ with $d_H(y, c') \leq t$, so c is the unique closest codeword to y. □

1.5.3 Geometric Intuition

The theorem has a natural geometric interpretation. Imagine placing a ball of radius t around each codeword in $\mathbb{F}_2^n$. The theorem says these balls are disjoint. If an error of weight at most t is introduced, the received word falls inside the ball around the true codeword, ensuring unique decoding.

More precisely, if the minimum distance is d, then the balls of radius $t = \lfloor (d-1)/2 \rfloor$ around each codeword do not overlap. To see why: if two balls around c and c' overlapped at some point y, then $d_H(c, c') \leq d_H(c, y) + d_H(y, c') \leq t + t = 2t < d$, contradicting the definition of minimum distance.

1.5.4 Decoding Algorithm

In practice, syndrome decoding proceeds as follows:

1. Receive $y = c + e$.
2. Compute the syndrome $s = Hy^T = He^T$.
3. Use a lookup table (or a decoding algorithm) to find the error vector e of minimum weight with the given syndrome.
4. Recover the codeword as $c = y + e$ and extract the message.

The key insight is that the syndrome uniquely identifies the error (assuming the error weight does not exceed t), and the decoder does not need to know the transmitted codeword or message beforehand.

1.5.5 Code Parameters and Trade-offs

A linear code is denoted $[n, k, d]$ with the following parameters:

- n is the codeword length.
- k is the dimension, so there are 2^k codewords.
- d is the minimum distance, determining the error-correction capability $t = \lfloor (d-1)/2 \rfloor$.

A good code achieves a balance: it should have a high rate k/n (to minimize redundancy), a large minimum distance d (to correct many errors), and short codeword length n (for practical efficiency). In reality, designing codes is often a trade-off between these competing goals.

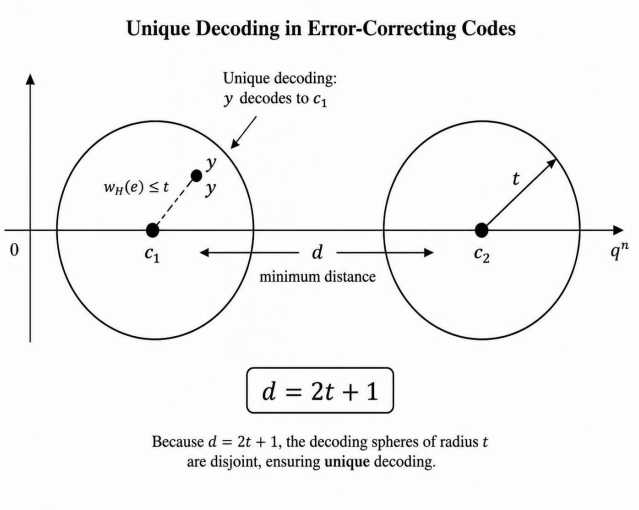


Figure 1.1: Unique decoding sphere visualization: if the minimum distance is d, then balls of radius $t = \lfloor (d-1)/2 \rfloor$ around each codeword are disjoint, ensuring unique decoding.

1.6 Systematic Forms of Codes

In practice, it is often useful to organize the generator matrix G in a *systematic form*, where the information from the message is directly visible in the codeword.

Definition 1.6.1. *A generator matrix G is in systematic form if it can be written as*

$$G = (I_k \mid P),$$

where I_k is the $k \times k$ identity matrix and P is a $k \times (n-k)$ matrix of parity bits. In other words,

$$G = \begin{pmatrix} 1 & 0 & \cdots & 0 & p_{1,1} & \cdots & p_{1,n-k} \\ 0 & 1 & \cdots & 0 & p_{2,1} & \cdots & p_{2,n-k} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & p_{k,1} & \cdots & p_{k,n-k} \end{pmatrix}.$$

1.6.1 Intuition: Information and Redundancy

When G is in systematic form, the encoding process becomes transparent. If the message is $m = (m_1, \ldots, m_k)$, then the codeword is

$$c = m \cdot G = (m_1, \ldots, m_k, p_1, \ldots, p_{n-k}),$$

where the first k positions are exactly the message, and the last $n-k$ positions are the parity bits computed as linear combinations of m.

Why is this useful?

- The information is directly embedded in the first k positions of the codeword — no need to "decode" the message from a complicated codeword structure.
- The parity bits (the matrix P part) are redundancy: they duplicate and check the information, allowing error detection and correction.
- At the receiver, after decoding, one simply extracts the first k bits to recover the original message.

1.6.2 Systematic Forms Always Exist

The key observation is that any linear code has a generator matrix, and any generator matrix can be transformed into systematic form using Gaussian elimination.

Proposition 1.6.2. *Any linear $[n, k]$ code has a generator matrix in systematic form.*

Proof. Let G be a $k \times n$ generator matrix for a linear code C. The rows of G are linearly independent (since they form a basis of C). Perform Gaussian elimination on G to reduce it to row-echelon form. Since the rows are linearly independent and there are k rows, we can reduce G to a form where:

1. The first k columns form an identity matrix I_k (the leading entries of each row are in different positions, forming a pivot structure).
2. The remaining columns contain arbitrary values.

By permuting columns if necessary (reordering the positions of the codeword), we can arrange the matrix so that the identity appears in the first k columns. This gives us $G = (I_k \mid P)$ in systematic form. □

Note: permuting columns changes the positions of information and parity in the codeword, but does not change the code itself or its properties.

1.6.3 Relationship with the Parity-Check Matrix

If $G = (I_k \mid P)$ is in systematic form, then a parity-check matrix in systematic form is

$$H = (-P^T \mid I_{n-k}) = (P^T \mid I_{n-k})$$

(over $\mathbb{F}_2$, where $-P = P$). This shows the duality between generator and parity-check matrices.

1.7 Bounds on Code Parameters

Upper bounds define the impossible frontier: they tell us what no code can achieve.

Theorem 1.7.1 (Singleton Bound)**.** *For any $[n, k, d]$ code over any alphabet:*

$$d \leq n - k + 1.$$

Proof. Let C be a linear $[n, k]$ code with minimum distance d. Puncture (remove) any $d-1$ coordinates. The resulting code C' has length $n - (d-1)$ and dimension at most k (puncturing cannot increase dimension). For C' to have distinct codewords, we must have $|C'| \leq 2^{n-(d-1)}$. But $|C'| = |C| = 2^k$, so:

$$2^k \leq 2^{n-d+1} \implies k \leq n - d + 1.$$

□

Codes achieving equality, $d = n - k + 1$, are called *Maximum Distance Separable (MDS)* codes and are extremely rare and valuable. For example, Reed Solomon Codes (see next chapter) achieve it.

Remark 1.7.2. *Singleton Bound means one cannot simultaneously maximize information rate (k/n) and error correction capability (d).*

Theorem 1.7.3 (Hamming (Sphere-Packing) Bound)**.** *If a code $C \subseteq \mathbb{F}_2^n$ corrects $t = \lfloor (d-1)/2 \rfloor$ errors, then:*

$$\sum_{i=0}^{t} \binom{n}{i} \leq 2^{n-k}.$$

Proof. If C corrects t errors, the Hamming balls $B_t(c)$ of radius t around each codeword are pairwise disjoint (by the triangle inequality: if $d(x, c_1) \leq t$ and $d(x, c_2) \leq t$ with $c_1 \neq c_2$, then $d(c_1, c_2) \leq 2t < d$, contradiction). Each ball has size $\sum_{i=0}^{t} \binom{n}{i}$. Since $|C| = 2^k$ such balls fit inside $\mathbb{F}_2^n$:

$$2^k \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n.$$

□

Perfect codes achieve equality in this bound — they waste no space. They are extremely rare (Hamming codes, Golay codes, and trivial cases).

Upper bounds tell us the ceiling. But they don't guarantee anything lives up there. Enter Gilbert-Varshamov.

Theorem 1.7.4 (Gilbert-Varshamov Bound)**.** *For any n and d, there exists a linear $[n, k, d]$ code with:*

$$k \geq n - \log_2 \sum_{i=0}^{d-2} \binom{n}{i}.$$

Proof. Construct a parity-check matrix H greedily. We need each nonzero vector in $\mathbb{F}_2^n$ to be expressible as a linear combination of at most $d-1$ columns of H. Start with an empty H and add columns one by one. As long as the number of such linear combinations is less than 2^{n-k} (the number of possible syndromes), we can always find a column that hasn't appeared yet. The number of forbidden linear combinations is:

$$\sum_{i=0}^{d-2} \binom{n}{i}.$$

If $\sum_{i=0}^{d-2} \binom{n}{i} < 2^{n-k}$, we can always add a new column. This gives $k \geq n - \log_2 \sum_{i=0}^{d-2} \binom{n}{i}$. □

Remark 1.7.5. *This is a constructive existence result: it guarantees that for any choice of (n, d), we can always build a code satisfying these parameters. Unlike the upper bounds, GV doesn't define an unattainable ideal — it shows what's always achievable.*

The open question

Upper bounds define what's impossible; GV shows what's always possible. The bounds are "relatively tight" — there's always a code somewhere in that feasible region, so we're not designing in a void.

But how tight are they really? GV tells us $k \geq \ldots$, and Hamming tells us $k \leq \ldots$. There is a gap between these two inequalities.

Can we construct codes that beat Gilbert-Varshamov while respecting the upper bounds? For specific families (like Hamming or BCH codes), yes — sometimes they outperform GV significantly. But in the general asymptotic regime, this gap remains largely open.

Closing this gap is one of the great problems in coding theory.

Chapter 2

Examples of Codes

2.1 Hamming Codes

2.1.1 The idea

We want a parity matrix H that tells us where an error has occurred. If e contains one non-zero bit at position i, $s = He^T$ will be the i-th column of H. In order to identify the column number, we construct H such that its i-th column is equal to i in binary. As H is an $m \times n$ matrix whose columns are all nonzero vectors, one can count at most $2^m - 1$ different column numbers. This is how we choose n: $n = 2^m - 1$, so that one column of H corresponds to one error position.

With this idea, a Hamming code is always $[2^m - 1, 2^m - m - 1]$. Finally, $d = 3$. If $w_H(c) = 1$ with 1 at position i, $Hc^T =$ column i, which is nonzero. If $w_H(c) = 2$ with 1s at positions i and j, $Hc^T =$ column $i\oplus$ column j, which is nonzero since all columns are distinct. However, when $w_H(c) = 3$ with 1s at positions i, j, l such that

$$\text{col}_i \oplus \text{col}_j = \text{col}_l$$

all XOR to 0. This means that a codeword of weight 3 exists.

Then, it is $[2^m - 1, 2^m - m - 1, 3]$. We call them SEC (Single Error Correction).

Remark 2.1.1. *Hamming codes are perfect codes. They achieve the sphere-packing bound with equality.*

Proof. Consider a Hamming code $[2^m - 1, 2^m - m - 1, 3]$ with minimum distance $d = 3$, so it corrects $t = \lfloor (d-1)/2 \rfloor = 1$ error. The sphere-packing bound states:

$$2^k \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n.$$

Substitute the Hamming parameters:

$$2^{2^m - m - 1} \cdot \sum_{i=0}^{1} \binom{2^m - 1}{i} \leq 2^{2^m - 1}.$$

Evaluate the binomial sum:

$$\sum_{i=0}^{1} \binom{2^m - 1}{i} = \binom{2^m - 1}{0} + \binom{2^m - 1}{1} = 1 + (2^m - 1) = 2^m.$$

Thus:

$$2^{2^m - m - 1} \cdot 2^m = 2^{2^m - m - 1 + m} = 2^{2^m - 1}.$$

The inequality becomes an equality, so Hamming codes are **perfect codes**: they achieve the maximum possible error-correction capability for their length and dimension. □

2.1.2 Example: the $[7, 4]$ Hamming code

Here is an example with the Hamming code $[7, 4, 3]$:

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Look at H's columns: $(001)_2 = 1$, $(010)_2 = 2$, ..., $(111)_2 = 7$.

A generator matrix can be derived from H, and after Gaussian elimination, one systematic form is:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} = (I_4 \mid P),$$

where P is the 4×3 parity matrix. A message (m_1, m_2, m_3, m_4) is encoded as

$$c = (m_1, m_2, m_3, m_4, m_1 + m_3 + m_4, m_2 + m_3 + m_4, m_1 + m_2 + m_3 + m_4),$$

where the last three bits are the parity checks.

Let us give an example of encoding and decoding with Hamming $[7, 4]$ code:

- Codeword: [1 0 1 1 0 1 0]
- Received: [1 1 1 1 0 1 1]

There are two errors, at positions 2 and 7. The syndrome is $2 \oplus 7 = 5$. Therefore, the syndrome is [1 0 1], which corresponds to position 5. The decoder assumes that there is one error at position 5 and flips that bit:

- Corrected word: [1 1 1 1 1 1 1]
- Decoded message: [1 1 1 1]

However, this correction is wrong. The original errors at positions 2 and 7 are still present, and flipping position 5 introduces a third error. A standard Hamming $[7, 4]$ code can correct one error, but it cannot reliably detect two errors.

2.1.3 SECDED Codes

An easy way to prevent this problem when two errors occur is by adding a global parity bit: take $[2^m, 2^m - m]$ instead of $[2^m - 1, 2^m - m - 1]$ and write

$$H = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

The matrix H has its first row full of 1s and columns permuted so G is in systematic form.

They are called SECDED (for Single Error Correction Double Error Detection). When $y = xG + e$ is received, the syndrome $s = Hy^T$ is then an m-dimensional vector and its first bit is the global parity check. Since the first row of H contains all 1s, this first bit is the XOR of all components in y. When y is a codeword c, we know that $Hc^T = 0$, meaning that there are no errors. This also implies that a valid codeword always has an even number of 1s. The global parity bit tells us exactly: the parity is OK if it equals 0, not OK otherwise. One error breaks the parity, while two errors restore it. These codes always assume that there are at most two errors, and cannot handle more.

Syndrome	Parity	Errors	Action
0	0	0	—
$\neq 0$	1	1	Corrects the one error
$\neq 0$	0	2	Detects 2 errors, no correction

Table 2.1: SECDED error correction behavior

Let's summarize:

See `src/hamming.py` for an implementation.

For instance, SECDED is pervasive in RAM and low-latency environments: it is computationally fast to decode and has negligible latency overhead, while remaining effective given that real-world memory errors rarely exceed two bits.

2.2 Reed-Solomon Codes

2.2.1 The principle

The idea is to see the message we want to transfer as a polynomial on a finite field $\mathbb{F}_q$ (often $\mathbb{F}_{2^m}$), with degree $< k$. In other words, the message is seen as k coefficients, each in $\mathbb{F}_q$. Choose $k < n < q$ different points from $\mathbb{F}_q$ known by the sender and the receiver. Evaluating the message-polynomial at these n points gives the codeword. We know that even with errors on some of these n points, k valid points allow to find back the polynomial by interpolation. This is the idea behind the information redundancy in RS codes. However, using Lagrange interpolation requieres to know the errors positions. An answer to this decoding problem is the Welch-Berlekamp algorithm, that is able to recover the original message-polynomial. The following describes the full process.

The message $m = (m_0, m_1, \ldots, m_{k-1})$ consists of k characters $m_0, \ldots, m_{k-1}$ of $\mathbb{F}_q$, seen as a polynomial

$$P(X) = m_0 + m_1 X + \cdots + m_{k-1} X^{k-1}.$$

Let us evaluate this polynomial on $n > k$ different points $\alpha_0, \alpha_1, \ldots, \alpha_{n-1} \in \mathbb{F}_q$:

$$c_i := P(\alpha_i), \qquad \text{for } 0 \leq i < n,$$

and the codeword is

$$c = (c_0, \ldots, c_{n-1}) = (P(\alpha_0), \ldots, P(\alpha_{n-1})).$$

In modern implementations, α_i's are often one primitive element from $\mathbb{F}_q$ to the powers i. In real life, encoding is done with the Vandermonde matrix : since

$$P(\alpha_i) = \sum_{j=0}^{k-1} m_j \alpha_i^j,$$

one can write

$$c = mG$$

with G the generating matrix $(k \times n)$ of the code, and the Vandermonde matrix

$$G = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \alpha_0 & \alpha_1 & \cdots & \alpha_{n-1} \\ \alpha_0^2 & \alpha_1^2 & \cdots & \alpha_{n-1}^2 \\ \vdots & \vdots & & \vdots \\ \alpha_0^{k-1} & \alpha_1^{k-1} & \cdots & \alpha_{n-1}^{k-1} \end{pmatrix},$$

so $G_{ji} = \alpha_i^j$.

Before we get into decoding methods, let's see how many errors it is capable of decoding.

Proposition 2.2.1. *The minimum distance of a Reed–Solomon code of length n and dimension k is*

$$d = n - k + 1.$$

Proof. Let

$$c = (P(\alpha_0), \ldots, P(\alpha_{n-1})) \quad \text{and} \quad c' = (Q(\alpha_0), \ldots, Q(\alpha_{n-1}))$$

be two distinct codewords, with $\deg(P), \deg(Q) < k$.

The nonzero polynomial $R = P - Q$ has degree at most $k - 1$. Therefore, it has at most $k - 1$ roots. The codewords c and c' coincide only at the points α_i that are roots of R. Hence, they coincide in at most $k - 1$ positions, and therefore

$$d_H(c, c') \geq n - (k - 1) = n - k + 1.$$

This bound is attained by taking

$$P(X) = \prod_{j=0}^{k-2} (X - \alpha_j) \quad \text{et} \quad Q(X) = 0.$$

The polynomial P has exactly $k - 1$ roots among the α_i. Consequently, the corresponding codewords differ in $n - k + 1$ positions. □

Reed-Solomon codes therefore reach the Singleton bound and are consequently MDS codes. They can thus decode up to

$$t = \lfloor (d - 1)/2 \rfloor = \lfloor (n - k)/2 \rfloor.$$

Suppose then that during the transmission, at most t errors occur.

The receiver gets

$$r_i = c_i + e_i, \qquad 0 \leq i \leq n - 1$$

with some characters r_i eventually corrupted.

First, in order to know whether errors occured or not, one can check the syndroms, defined as

$$S = H \cdot r^T.$$

with $r = (r_0, \ldots, r_{n-1})$. If all the $n - k$ coordinates of S are zero, there is no error, since every valid codeword satisfies $Hc^T = 0$.

Else, the goal is to find P without knowing the errors positions in advance. We introduce an error-localizing polynomial:

$$E(X) = \prod_{i \in \mathcal{E}} (X - \alpha_i),$$

where $\mathcal{E}$ is the set of errors positions. We dont know $\mathcal{E}$ yet, so we write

$$E(X) = 1 + E_1 X + \cdots + E_t X^t$$

with unknown coefficients. By definition,

$$E(\alpha_i) = 0, \qquad \text{for } i \in \mathcal{E}.$$

Then, we introduce

$$Q(X) = E(X)P(X) = q_0 + q_1 X + \cdots + q_{k+t-1} X^{k+t-1},$$

defined such as

$$Q(\alpha_i) = r_i E(\alpha_i)$$

for each character (α_i, r_i) received. Developping, we get:

$$q_0 + q_1\alpha_i + \cdots + q_{k+t-1}\alpha_i^{k+t-1} = r_i\left(1 + E_1\alpha_i + \cdots + E_t\alpha_i^t\right)$$

Placing the unknowns (E_i's and q_i's) left:

$$-r_i\alpha_i E_1 - r_i\alpha_i^2 E_2 - \cdots - r_i\alpha_i^t E_t + q_0 + q_1\alpha_i + \cdots + q_{k+t-1}\alpha_i^{k+t-1} = r_i.$$

Finally, putting the n equations together we get the following linear system:

$$\underbrace{\begin{pmatrix} -r_0\alpha_0 & -r_0\alpha_0^2 & \cdots & -r_0\alpha_0^t & 1 & \alpha_0 & \cdots & \alpha_0^{k+t-1} \\ -r_1\alpha_1 & -r_1\alpha_1^2 & \cdots & -r_1\alpha_1^t & 1 & \alpha_1 & \cdots & \alpha_1^{k+t-1} \\ \vdots & \vdots & & \vdots & \vdots & \vdots & & \vdots \\ -r_{n-1}\alpha_{n-1} & -r_{n-1}\alpha_{n-1}^2 & \cdots & -r_{n-1}\alpha_{n-1}^t & 1 & \alpha_{n-1} & \cdots & \alpha_{n-1}^{k+t-1} \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} E_1 \\ E_2 \\ \vdots \\ E_t \\ q_0 \\ q_1 \\ \vdots \\ q_{k+t-1} \end{pmatrix}}_{x} = \underbrace{\begin{pmatrix} r_0 \\ r_1 \\ \vdots \\ r_{n-1} \end{pmatrix}}_{b}.$$

Once the system is solved, we know explicitely

$$E(X) = 1 + \sum_{j=1}^{t} E_j X^j, \qquad Q(X) = \sum_{j=0}^{k+t-1} q_j X^j,$$

so we can find

$$P(X) = \frac{Q(X)}{E(X)}.$$

With its coefficients, the message is decoded.

This decoding method costs the solving of the linear system which is $O(n \times (k + 2t)) \simeq O(n^3)$.

A second decoding approach is based on syndromes and the Berlekamp–Massey algorithm. Although this method is more efficient than Welch–Berlekamp decoding, it is somewhat more difficult to understand and is less directly connected to the definition of Reed–Solomon codes. This is the approach implemented by the `galois.ReedSolomon` class which costs $O(t^2)$ elementary operations.

2.2.2 A small Reed–Solomon example

Let's work over the finite field $\mathbb{F}_{11}$ and take $n = 5$, $k = 3$. We evaluate the message polynomial at the points

$$\alpha_0 = 0, \quad \alpha_1 = 1, \quad \alpha_2 = 2, \quad \alpha_3 = 3, \quad \alpha_4 = 4.$$

Since $d = n - k + 1 = 3$, the code can correct $t = \left\lfloor \frac{d-1}{2} \right\rfloor = \left\lfloor \frac{2}{2} \right\rfloor = 1$ error. Let the message be

$$m = (2, 3, 1) \in \mathbb{F}_{11}^3.$$

The associated polynomial is

$$P(X) = 2 + 3X + X^2.$$

The codeword is obtained by evaluating P at the five chosen points:

$$\begin{aligned} P(0) &= 2, \\ P(1) &= 2 + 3 + 1 = 6, \\ P(2) &= 2 + 6 + 4 = 12 \equiv 1 \pmod{11}, \\ P(3) &= 2 + 9 + 9 = 20 \equiv 9 \pmod{11}, \\ P(4) &= 2 + 12 + 16 = 30 \equiv 8 \pmod{11}. \end{aligned}$$

Therefore, the encoded word is

$$c = (2, 6, 1, 9, 8).$$

Suppose that the symbol corresponding to $\alpha_2 = 2$ is corrupted. For example, let an error of value 4 occur at this position. The receiver then obtains

$$r = (2, 6, 5, 9, 8).$$

He can check if errors occured. He knows

$$G = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \\ 0 & 1 & 4 & 9 & 5 \end{pmatrix}, \qquad H = \begin{pmatrix} 1 & 0 & 5 & 8 & 8 \\ 0 & 1 & 8 & 3 & 10 \end{pmatrix},$$

so $S = Hr^T = (9, 10)^T$ which is not zero. We now apply the Welch–Berlekamp decoding method. Since at most one error is expected, we write the error-locator polynomial as

$$E(X) = 1 + E_1 X$$

and define

$$Q(X) = E(X)P(X).$$

Since $\deg(E) = 1$ and $\deg(P) < 3$, we have $\deg(Q) \leq 3$. Write

$$Q(X) = q_0 + q_1 X + q_2 X^2 + q_3 X^3.$$

The unknowns are $E_1,\ q_0,\ q_1,\ q_2,\ q_3$. Using the relation

$$Q(\alpha_i) = r_i E(\alpha_i)$$

for each received symbol, we obtain

$$-r_i \alpha_i E_1 + q_0 + q_1 \alpha_i + q_2 \alpha_i^2 + q_3 \alpha_i^3 = r_i,$$

and for the received word $r = (2, 6, 5, 9, 8)$, this gives the following linear system over $\mathbb{F}_{11}$:

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 5 & 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 4 & 8 \\ 6 & 1 & 3 & 9 & 5 \\ 1 & 1 & 4 & 5 & 9 \end{pmatrix} \begin{pmatrix} E_1 \\ q_0 \\ q_1 \\ q_2 \\ q_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 6 \\ 5 \\ 9 \\ 8 \end{pmatrix} \qquad \text{in } \mathbb{F}_{11}.$$

Solving this system gives

$$\begin{pmatrix} E_1 \\ q_0 \\ q_1 \\ q_2 \\ q_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \\ 2 \\ 5 \\ 5 \end{pmatrix}.$$

Therefore,

$$E(X) = 1 + 5X$$

and

$$Q(X) = 2 + 2X + 5X^2 + 5X^3.$$

Its root is

$$1 + 5X = 0 \quad \Longleftrightarrow \quad X = 2 \quad \text{in } \mathbb{F}_{11},$$

which correctly identifies the error position $\alpha_2 = 2$.

Finally, we divide $Q(X)$ by $E(X)$:

$$P(X) = \frac{Q(X)}{E(X)} = \frac{2 + 2X + 5X^2 + 5X^3}{1 + 5X}.$$

Over $\mathbb{F}_{11}$, this gives

$$P(X) = 2 + 3X + X^2.$$

The original message is therefore recovered as

$$m = (2, 3, 1).$$

See `src/reed_solomon.py` for an implementation.

Today, Reed–Solomon codes are still widely used in QR codes and two-dimensional barcodes, optical storage media such as CDs, DVDs and Blu-ray discs, distributed storage systems and RAID-like erasure coding, as well as in several digital broadcasting and satellite communication standards or spatial communications. Their main advantage is their ability to correct symbol errors and erasures, especially when errors occur in bursts. However, newer communication systems such as 5G mainly rely on LDPC and polar codes, which are better suited to their specific performance requirements.



Figure 2.1: Reed-Solomon codes have enabled the Hubble Telescope to transmit images like this back to scientists on Earth. The galaxy depicted in this Hubble image is a barred spiral galaxy, known as NGC 7541, in the constellation of Pisces. Photo courtesy of NASA, source : MIT, [1]

2.3 Low Density Parity Check (LDPC) Codes

2.3.1 Definition and Bit-Flipping Algorithm

Low-Density Parity-Check (LDPC) codes form a large class of linear codes defined by a parity-check matrix $H \in \mathbb{F}_2^{(n-k)\times n}$ of low density. This sparsity is not merely incidental: it enables efficient iterative decoding at linear complexity, in sharp contrast to standard algorithms which scale exponentially like `Prange` (see the random codes section).

A parity-check matrix H is called *sparse* if the number of ones per row (or per column) is $O(1)$ or $O(\log n)$, independent of the code length n. More precisely, if H has $(n - k)$ rows and n columns, a dense matrix would contain $\Theta(n)$ ones per row, whereas a sparse LDPC matrix contains only d_c

or d_v ones (the column and row weights, respectively), with $d_c, d_v \ll n$ typically fixed or growing logarithmically. For example, in a practical LDPC code of length $n = 10^6$, a sparse parity matrix might have only $d_c \approx 6$ ones per column and $d_v \approx 60$ ones per row, meaning the matrix density is roughly $6/n \approx 10^{-6}$.

A codeword $c \in \mathbb{F}_2^n$ satisfies $Hc^T = 0$ modulo 2. This constraint decomposes into $(n-k)$ independent linear equations. Crucially, when H is sparse, each parity constraint $h_i \cdot c^T = 0$ involves only a small number of bit positions—namely, d_c positions corresponding to the ones in row i. Consequently, errors corrupt parity constraints locally, permitting decentralized error detection and localization.

Upon transmission, the received vector $r = c + e$ produces a syndrome $s = Hr^T = He^T \in \mathbb{F}_2^{n-k}$. The syndrome encodes information about the error pattern e. A simple decoding method is called `Bit-Flipping`. The Bit-Flipping algorithm operates iteratively as follows: at each iteration, for every bit position j, count the number of violated parity checks that involve position j. Flip the bits at positions with the highest violation counts, then recompute the syndrome. This approach leverages the sparse structure: each bit position appears in only d_v constraints, making decoding polynomial in n, with complexity $O(n \cdot d_v \cdot \text{iterations})$.

Unlike Hamming or Reed-Solomon codes, LDPC codes lack a generic guarantee on minimum distance. Consequently, there is no universal error-correction capability bound t such that Bit-Flipping corrects all error patterns of weight $\leq t$. Nevertheless, well-designed LDPC codes (typically constructed via numerical optimization) possess remarkable decoding thresholds: they decode successfully with overwhelming probability as long as the channel error rate remains below a critical threshold, asymptotically approaching the Shannon limit. This empirical advantage, despite the absence of worst-case guarantees, makes LDPC codes among the most practical error-correcting codes for modern communication systems.

Let us describe the Bit-Flipping decoding algorithm :

Bit-Flip Decoding Algorithm

Input: Received vector $r \in \mathbb{F}_2^n$, parity-check matrix $H \in \mathbb{F}_2^{(n-k)\times n}$, maximum number of iterations $I_{\max}$

Output: Estimated codeword $\hat{c}$

Initialize: $\hat{c} \leftarrow r$

For iter $= 1$ to $I_{\max}$:

1. Compute syndrome: $s \leftarrow H\hat{c}^T \pmod 2$
2. If $s = 0$: return $\hat{c}$ (decoding success)
3. Else, for each bit position $j \in \{0, 1, \ldots, n-1\}$:

Count violations involving position j: $\texttt{count}[j] \leftarrow \#\{i : H_{i,j} = 1 \text{ and } s_i = 1\}$

4. Find position $j^* = \arg\max_j \texttt{count}[j]$
5. Flip the bit $\hat{c}[j^*]$

Return $\hat{c}$ (decoding failure if $s \neq 0$)

Note that this algorithm is heuristic : it may or may not return the right codeword, depending on the structure of the code. Also, excracting the message from $\hat{c}$ becomes immediate when the code is in systematic form.

2.3.2 Example 1: Bit-Flipping that succeeds

LDPC Code (6, 3) Parity-check matrix (systematic form):

$$H = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Generator matrix (systematic form):

$$G = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Encoding Message: $m = (1, 0, 1)$

Codeword: $\hat{c} = m \cdot G = (1, 0, 1, 0, 1, 1)$ (the first 3 bits are the message, the last 3 are parity bits)

Transmission with 1 Error Received: $r = (1, 1, 1, 0, 1, 1)$ (error at bit position 1)

Bit-Flipping Decoding Iteration 1:

- **Syndrome**:

$$s = H \cdot r^T \pmod 2 = \begin{pmatrix} 1+1+1+0 \\ 1+1+0+1 \\ 1+1+0+1 \end{pmatrix} \pmod 2 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

  $s \neq 0 \rightarrow$ continue

- **Violation count for each position**:
  - Position 0: involved in rows $\{0, 2\}$ (where $H_{0,0} = 1, H_{2,0} = 1$); violations among $\{0, 2\}$ (where $s_i = 1$): $\{0, 2\} \rightarrow \texttt{count}[0] = 2$
  - Position 1: involved in rows $\{0, 1, 2\}$; violations: $\{0, 1, 2\} \rightarrow \texttt{count}[1] = 3$ **MAX**
  - Position 2: involved in rows $\{0, 1\}$; violations: $\{0, 1\} \rightarrow \texttt{count}[2] = 2$
  - Position 3: involved in rows $\{0\}$; violations: $\{0\} \rightarrow \texttt{count}[3] = 1$
  - Position 4: involved in row $\{1\}$; violations: $\{1\} \rightarrow \texttt{count}[4] = 1$
  - Position 5: involved in row $\{2\}$; violations: $\{2\} \rightarrow \texttt{count}[5] = 1$
- **Flip max position**: $j^* = 1$ (choose the first in case of tie)

$$\hat{c} \leftarrow (1, 0, 1, 0, 1, 1)$$

Iteration 2:

- **Syndrome**:

$$s = H \cdot (1, 0, 1, 0, 1, 1)^T = \begin{pmatrix} 1+0+1+0 \\ 0+0+1+0 \\ 1+0+0+1 \end{pmatrix} \pmod 2 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

  $s = 0 \rightarrow$ Success

Recovered Message The first 3 bits: $(1, 0, 1)$

2.3.3 Example 2: Bit-Flipping that Fails

Transmission with 4 Errors Error vector: $e = (1, 1, 1, 0, 1, 0)$ at positions 0, 1, 2, 4

Codeword: $\hat{c} = (1, 0, 1, 0, 1, 1)$

Received: $r = (0, 1, 0, 0, 0, 1)$

Bit-Flipping Decoding Iteration 1:

- **Syndrome**:

$$s = H \cdot r^T \pmod 2 = \begin{pmatrix} 0+1+0+0 \\ 1+0+0+0 \\ 0+1+0+1 \end{pmatrix} \pmod 2 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

- **Violation count**: $\texttt{count} = [1, 2, 2, 1, 1, 0]$
- **Flip position**: $j^* = 1$

$$\hat{c} \leftarrow (0, 0, 0, 0, 0, 1)$$

- **New syndrome**:

$$s = (0, 0, 1)$$

Iteration 2:

- **Violation count**: $\texttt{count} = [1, 1, 0, 0, 0, 1]$
- **Flip position**: $j^* = 0$

$$\hat{c} \leftarrow (1, 0, 0, 0, 0, 1)$$

- **New syndrome**:

$$s = (1, 0, 0)$$

Iteration 3:

- **Violation count**: $\texttt{count} = [1, 1, 1, 1, 0, 0]$
- **Flip position**: $j^* = 0$ (tie; chose first)

$$\hat{c} \leftarrow (0, 0, 0, 0, 0, 1)$$

- **New syndrome**:

$$s = (0, 0, 1)$$

Result The algorithm enters an **infinite loop** of period 2:

$$(0, 0, 0, 0, 0, 1) \xrightarrow{s=(0,0,1)} (1, 0, 0, 0, 0, 1) \xrightarrow{s=(1,0,0)} (0, 0, 0, 0, 0, 1) \cdots$$

With those 4 simultaneous errors, the error lies outside the decodable region (typically $|e| < d_{\min}/2$). Bit-Flipping fails to converge.

See `src/ldpc.py` for an implementation.

In these small pedagogical examples ($n = 6$), the matrix H is very dense (roughly 55% of ones). However, this is misleading. In practical LDPC codes (for instance, $n = 10^6$), H is constructed with sparsity: the number of ones per row and per column remains constant (typically $O(1)$ or $O(\log n)$), maintaining the total density at a vanishingly small fraction (for example, $\approx 10^{-5}\%$).

It is precisely this sparsity that enables decoding in linear time: $O(n \cdot d_v \cdot \text{iterations})$. Without it, we would be forced to resort to exponential-time algorithms such as Prange's algorithm (that decodes random codes), whose complexity renders decoding infeasible for large code dimensions.

As shown in this last example, the bit-flipping algorithm may often fail. An improvement of this algorithm is presented in the next chapter. Other decoding algorithms like the Sum-Product Algorithm or the Min-Sum Algorithm are sometimes used for better decoding capabilities, but with higher computational cost per iteration.

Here are two graphs swhowing the decoding success rate depending on the length of the code and on the error rate.

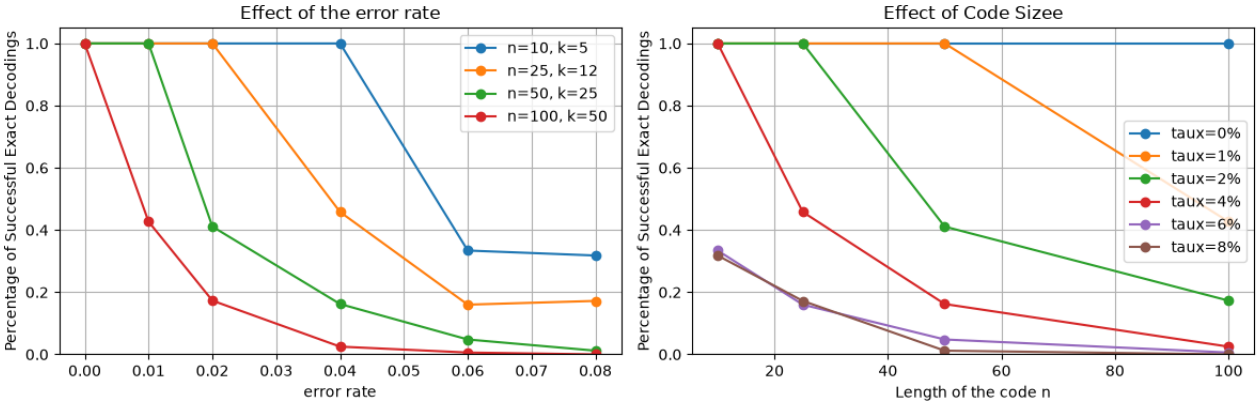


Figure 2.2: Bit-Flipping Successful Decoding Rate

LDPC codes are now deployed across numerous practical systems:

- 5G and beyond: Used in the 5G NR (New Radio) standard for channel coding.
- WiFi and WiMAX: IEEE 802.11n and IEEE 802.16e standards employ LDPC codes.
- Satellite and deep-space communication: NASA missions rely on LDPC codes for reliability over extremely noisy channels.
- Streaming video and data centers: Used to protect against burst errors in storage and transmission.
- Quantum error correction: LDPC-like constructions inspire quantum error-correcting codes.
- Cryptography : The BIKE cryptosystem (see next chapter).

2.4 Random Codes

Chapter 3

Error-Correcting Codes in Cryptography

3.1 Motivation

Modern Cryptography faces the incoming quantum computer that breaks asymetric primitives like RSA, Diffie-Hellman, Elliptic Curves,... all used currently for key exchange and digital signatures. Error-Correcting Codes give a natural way to create some quantum-resistant key exchange cryptosystems.

3.2 BIKE Scheme

Bibliography

[1] MIT Lincoln Laboratory. Empowering digital communications. URL: https://www.ll.mit.edu/impact/empowering-digital-communications.